

Personal Research Journal

Author: Yiming Lu

Header & Context

Timestamp & Entry Title: October 1, 2026 - ROS2 Cartographer IMU Integration, TF Configuration, and Sensor Validation

Project Name/Code: F1TENTH Autonomous Vehicle - ROS2 SLAM / Cartographer IMU Integration; workspace: ~/newf1_ws

Current Hypothesis/Goal: Integrate the VESC IMU with Cartographer, establish the required TF relationships between base_link, imu, and laser, and verify that LiDAR and IMU data can be used simultaneously for SLAM.

Methodology & Execution

Experimental / Computational Setup

The experiments were performed on the F1TENTH vehicle using ROS2 Humble. The main hardware involved was a Hokuyo LiDAR and a VESC-based IMU. The main packages used were vesc_driver, urg_node, tf2_ros, my_robot_bringup, my_cartographer_config, and cartographer_ros.

BASH

```
# Sensor configuration
Hokuyo LiDAR
topic: /scan
type: sensor_msgs/msg/LaserScan
frame: laser
expected frequency: ~10 Hz

VESC IMU
topic: /sensors/imu/raw
type: sensor_msgs/msg/Imu
frame: imu
expected frequency: ~50 Hz
```

Exact Code, Scripts, or Commands

BASH

```
$ source ~/newf1_ws/install/setup.bash
$ ros2 run tf2_ros tf2_echo base_link imu
$ ros2 topic hz /scan
$ ros2 topic hz /sensors/imu/raw
$ ros2 topic info /scan --verbose
$ ros2 node info /urg_node
$ ps aux | grep -E "urg_node|urg_node_driver"
$ ros2 launch my_robot_bringup robot.launch.py
$ ros2 launch my_cartographer_config cartographer.launch.py
$ ros2 node info /cartographer_node
$ ros2 run tf2_ros tf2_echo map base_link
$ ros2 topic list | grep map
$ ros2 topic hz /map
$ rviz2
```

Detailed Steps Taken

1. Verification of the IMU TF

I first checked the transform between base_link and imu. ROS2 initially reported that the base_link frame did not exist. Inspection of ~/newf1_ws/src/my_robot_bringup/launch/robot.launch.py showed that the static base_link -> imu transform had already been defined, but its node was

commented out in the returned LaunchDescription. I enabled `static_tf_base_to_imu`, rebuilt the package, and relaunched it.

BASH

```
$ cd ~/newf1_ws
$ colcon build --packages-select my_robot_bringup
$ source install/setup.bash
```

2. Sensor publishing-rate validation

The VESC IMU was measured at approximately 50 Hz with message intervals around 0.015-0.025 s. The first LiDAR test was abnormal, with rates initially around 0.06-0.17 Hz and gaps as large as 24.432 s.

3. Isolation of the LiDAR problem

`ros2 topic info /scan --verbose` showed a publisher count of 2, and ROS2 reported two nodes with the exact name `/urg_node`. A process-level check confirmed two `urg_node_driver` processes. One had been started manually, while `robot.launch.py` started another. I stopped the manually launched instance.

BASH

```
$ ros2 topic info /scan --verbose
Publisher count: 2

$ ros2 node info /urg_node
There are 2 nodes in the graph with the exact name "/urg_node".

$ ps aux | grep -E "urg_node|urg_node_driver"
# Two urg_node_driver processes were present.
```

4. Cartographer IMU configuration

I edited `~/newf1_ws/src/my_cartographer_config/config/hokuyo_2d.lua`, changed the tracking frame to `imu`, retained `published_frame = "base_link"`, and kept IMU usage enabled.

LUA

```
tracking_frame = "imu",
published_frame = "base_link",
TRAJECTORY_BUILDER_2D.use_imu_data = true
```

I also verified the remapping in

`~/newf1_ws/src/my_cartographer_config/launch/cartographer.launch.py`.

PYTHON

```
remappings=[
    ('scan', '/scan'),
    ('imu', '/sensors/imu/raw')
]
```

BASH

```
$ cd ~/newf1_ws
$ colcon build --packages-select my_cartographer_config
$ source install/setup.bash
```

5. Cartographer launch and verification

After launching Cartographer, `ros2 node info /cartographer_node` confirmed subscriptions to both sensor topics. The map `-> base_link` transform became available after startup and then updated continuously.

BASH

```
$ ros2 node info /cartographer_node

Subscribers:
  /scan: sensor_msgs/msg/LaserScan
  /sensors/imu/raw: sensor_msgs/msg/Imu
```

6. Occupancy-grid output and RViz preparation

The topic /map was present and published at a stable 1.000 Hz. RViz2 was opened to begin visual validation using Fixed Frame = map, Map = /map, LaserScan = /scan, and TF visualization.

Results & Observations

Measurement / condition	Observed result	Interpretation
VESC IMU	~50 Hz	Stable
LiDAR with duplicate urg_node	~0.06-0.17 Hz initially	Abnormal
LiDAR after duplicate removed	~10.00 Hz	Stable
/map occupancy grid	1.000 Hz	Stable

Raw Data & Outputs

```
OUTPUT

# IMU
average rate: 49.993 Hz
average rate: 50.001 Hz
average rate: 49.999 Hz

# LiDAR after troubleshooting
average rate: 10.002 Hz
average rate: 10.005 Hz
average rate: 10.003 Hz
min: ~0.095-0.096 s
max: ~0.104-0.105 s

# Occupancy grid
/map average rate: 1.000 Hz
```

Unexpected Behavior

The major unexpected behavior was the extremely low and intermittent LiDAR publishing rate. The issue was traced to two Hokuyo driver processes attempting to access the same sensor. Another issue was that the IMU static transform existed in the source launch file but was not active because it was commented out of the launch description. Both problems were corrected without changing the LiDAR hardware.

Visual Artifacts

RViz2 was opened for visual inspection. At the end of this week's session, the displays were being configured so that the occupancy grid uses /map, the LaserScan display uses /scan, TF is enabled, and the fixed frame is map.

Analysis & Reflection

Data Interpretation

The main ROS communication objective was achieved. The VESC IMU publishes steadily at approximately 50 Hz, the Hokuyo LiDAR at approximately 10 Hz, Cartographer subscribes to both sensors, a continuously updating map -> base_link transform is produced, and the occupancy grid is published at 1 Hz. These results establish that the basic LiDAR-IMU SLAM data pipeline is

operational, although motion-based validation is still required.

Troubleshooting & Deductions

The LiDAR failure demonstrated that abnormal sensor-topic timing can be caused by ROS process configuration rather than by the physical sensor. Checking publisher count, duplicate node names, and operating-system processes isolated the cause. The TF issue similarly showed that defining a transform node in source code is insufficient unless that node is included in the active launch description.

Literature Connections

The work follows the Cartographer ROS configuration model in which sensor streams are supplied through ROS topics and connected through a TF tree. ROS2 tf2 documentation, the sensor_msgs definitions for Imu and LaserScan, and Cartographer ROS configuration documentation remain relevant references.

Next Steps & Open Questions

Immediate Action Items

1. Complete RViz visualization: configure Fixed Frame = map, Map = /map, and LaserScan = /scan, then verify that LiDAR points align with the generated occupancy grid.
2. Perform a controlled motion test: slowly translate and rotate the F1TENTH vehicle while observing the Cartographer pose and map, and verify the IMU yaw direction and TF alignment.
3. Measure ego-vehicle speed in real time: identify an appropriate real-time vehicle-speed source (for example, VESC/odometry data), publish or extract the ego-car velocity in ROS2, and validate the measured speed against controlled vehicle motion. This can later support motion analysis, SLAM evaluation, and autonomous-control experiments.

Unresolved Questions

The remaining questions are whether the current identity rotational transform between base_link and imu accurately represents the physical mounting orientation; how much pose drift occurs while stationary and during motion; whether enabling the IMU measurably improves Cartographer compared with LiDAR-only operation; which ROS2 signal provides the most reliable ego-vehicle speed estimate; and whether Cartographer parameters will require tuning at realistic F1TENTH driving speeds.